

ZEPPLY

Core Product Specification

Create. Reply. Multiply.

The always-on AI marketing platform for creators and businesses

Audience	Zepply engineering team (developers building the platform)
Scope	Web app (primary) + iOS/Android app: product, architecture, data model, module specs, integrations, roadmap
Detail level	Wave 1 (shared core) fully specified · Waves 2–4 specified at feature level
Status	v1.0 — living document; open decisions listed in Section 12

Contents

- 1. How to use this document 4
- 2. Product overview 4
 - 2.1 The problem 4
 - 2.2 Who it is for 5
 - 2.3 The always-on loop: Create → Reply → Multiply 5
 - 2.4 Positioning and differentiation 5
- 3. Product principles 6
- 4. Product structure 6
 - 4.1 Module map 6
 - 4.2 Plans and entitlements 7
 - 4.3 How the creator / business choice changes the app 7
- 5. System architecture 8
 - 5.1 Overview 8
 - 5.2 Recommended stack 8
 - 5.3 Architecture rules 9
 - 5.4 Web and mobile apps 9
 - 5.5 What each app does 10
 - 5.6 Mobile-specific requirements 10
 - 5.7 Rules that keep web and mobile in sync 11
- 6. Data model 11
 - 6.1 Entities 11
 - 6.2 Content item lifecycle 12
- 7. Wave 1 — shared core module specs 13
 - M1 Onboarding and workspaces 13
 - M2 Brand kit from a link 14
 - M3 Content studio — AI posts, stories and calendar 14
 - M4 Clip engine — long-form to short-form 15
 - M5 Reels generator and AI video editor 16
 - M6 Publisher and scheduler 16
 - M7 Instagram DM automation 17
 - M8 Growth analytics 17
 - M9 Credits, plans and billing 18

- M10 Approval queue and autopilot 18
- 8. Business layer specs (Waves 2–4) 19
 - B1 · Product photoshoots and business photos/videos (Wave 2) 19
 - B2 · WhatsApp automation and unified inbox (Wave 2) 19
 - B3 · Leads and contacts (Wave 2) 19
 - B4 · Ads engine: launch, campaign analysis, learning loop (Wave 3) 19
 - B5 · AI voice agent add-on (Wave 3) 19
 - B6 · Website, Google Business Profile, reviews (Wave 4) 20
- 9. Integrations and external dependencies 20
 - 9.1 Platform permissions 20
 - 9.2 AI and media providers 20
- 10. Non-functional requirements 21
 - 10.1 Performance targets 21
 - 10.2 Security 21
 - 10.3 Privacy and compliance 21
 - 10.4 Cost control 21
 - 10.5 Languages and text rendering 22
 - 10.6 Reliability and observability 22
- 11. Roadmap and success metrics 22
 - 11.1 Waves and exit criteria 22
 - 11.2 Metrics to instrument from day one 22
- 12. Existing assets and open decisions 23
 - 12.1 What already exists and should be reused 23
 - 12.2 Open decisions 23
- 13. Glossary 24

SECTION 01

1. How to use this document

This is the reference for building Zepply. It explains what we are building, for whom, the rules every feature must follow, and how the system fits together. Read Sections 2–4 before writing any code; they define decisions that every module depends on.

Section	Read it when you need to...
2 · Product overview	Understand the product, the two audiences and how the always-on loop works
3 · Product principles	Make any product or UX decision — these rules override feature requests
4 · Product structure	See which modules exist and which plan / audience gets them
5 · System architecture	Choose where code lives, how services talk, how jobs run, what web vs. mobile does
6 · Data model	Design tables, APIs and state machines
7 · Wave 1 module specs	Build the shared core (fully specified, with acceptance criteria)
8 · Business layer specs	Plan Waves 2–4
9 · Integrations	Work with Meta, WhatsApp, AI providers; start long-lead approvals
10 · Non-functional requirements	Security, performance, cost control, compliance, languages
11 · Roadmap and metrics	Know what ships when and how success is measured
12–13 · Reuse, decisions, glossary	Find existing code to reuse and questions still open

Words used precisely in this document

MUST = required for the feature to ship. **SHOULD** = expected unless there is a written reason not to. **MAY** = optional. **Workspace** = one creator or one business account (a tenant). **Credit** = the unit users spend on AI generation.

SECTION 02

2. Product overview

Zepply is a full, always-on AI marketing platform. Tagline: “Create. Reply. Multiply.” A creator or business connects their accounts once; Zepply learns the brand, creates content, publishes it on schedule, replies to people who engage, measures what worked, and uses those results to make the next round better — continuously, without the owner having to drive it.

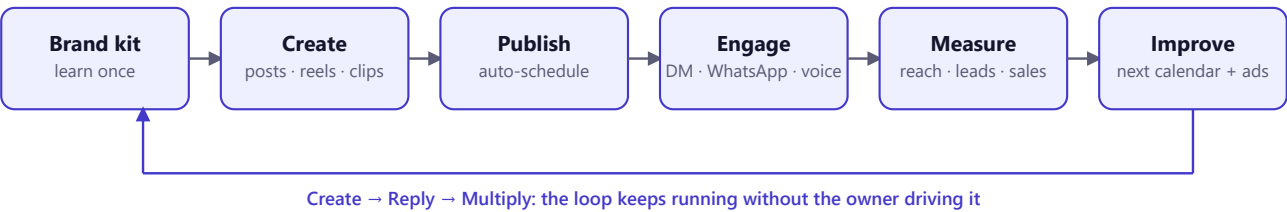
2.1 The problem

- Creators and small businesses must post daily, reply to every DM and run ads — work that normally needs a social media manager, a video editor, a designer and an ads specialist.
- Existing tools each solve one slice (clipping, design, DM bots, WhatsApp tools, ad managers). Users juggle 4–6 subscriptions and nothing learns across them.
- Most AI tools handle Indian languages poorly, especially code-mixed speech such as Telugu-English and Hinglish.
- AI content tools lose user trust through avoidable failures: altering the real product in photos, charging again to fix the AI’s own mistakes, and spending credits without asking. (Observed in a direct competitor with a 2.8-star Play Store rating — see Section 3.)

2.2 Who it is for

	Creator	Business
Who	Influencers, podcasters, educators, coaches, personal brands	D2C brands, local businesses (salons, clinics, restaurants, gyms), real estate, service businesses
Core job	Grow an audience by posting consistently and engaging with followers	Turn marketing into leads and sales
Needs	AI posts, long-form→shorts clipping, Reels, AI video editing, follower growth, content generation, Instagram DM automation	Everything a creator needs, plus ad campaigns, business photos/videos, product photoshoots, campaign analysis, WhatsApp automation
Headline metric	Followers, reach, engagement	Leads, cost per lead, return on ad spend (ROAS)

2.3 The always-on loop: Create → Reply → Multiply



- **Create:** a content calendar is always filled at least 14 days ahead.
- **Publish:** approved content goes out at the best time for that audience, without the user opening the app.
- **Engage:** comments, DMs and WhatsApp messages get an answer within 60 seconds, day or night, in the sender’s language.
- **Measure and improve:** results feed back into the next calendar and the next ad campaign automatically.

2.4 Positioning and differentiation

Tagline: “Create. Reply. Multiply.” — *Create* = posts, Reels, clips, photoshoots; *Reply* = Instagram DMs, WhatsApp, voice; *Multiply* = followers, leads, sales. Do not use “24/7” wording in product copy (a competitor’s headline). Creators hear: “*Post every day without burning out — clips, Reels and DMs on autopilot.*” Businesses hear: “*From content to customers — content, ads and every lead answered in your language.*”

Our edge	Why it is hard to copy
One connected loop (content → publish → engage → measure → improve)	Competitors own single slices; only a connected system can learn from outcomes
Code-mixed Indian languages (Telugu, Hindi, English and mixes)	Needs tuned transcription, alignment, caption shaping and hook rubrics — we already have this in the clip engine
AI voice agent	Already built and in client demos; no content-tool competitor has it
Trust-first product rules (Section 3)	A cultural choice competitors have not made; becomes a reputation advantage

SECTION 03

3. Product principles

These rules apply to every module. If a feature request conflicts with a principle, the principle wins until the product owner changes it in writing.

Principle	Rule
P1 · Never alter the real product	Product photos and videos MUST keep the actual product pixel-faithful. Cut the product out and composite it onto generated backgrounds; never regenerate the product itself. For real estate, never invent or change property features (misleading property ads can breach RERA rules).
P2 · Edit, don't regenerate	Every output MUST be editable in parts — a caption line, a scene, a clip boundary, a background. Users fix small things without paying for a whole new generation.
P3 · If the AI got it wrong, the redo is free	A user can mark an output as “AI mistake” (wrong product, missed instruction, broken text). The redo MUST NOT cost credits. Abuse is controlled by limits (Section 7, M9), not by charging.
P4 · Ask before spending	Every action that spends credits MUST show the cost and require confirmation. No silent deductions — including “preview” or “try it” actions.
P5 · Human approval by default; autopilot is earned	New workspaces start with everything going through an approval queue. Users switch on autopilot per content type once they trust it.
P6 · Platform-safe growth only	No fake followers, follow/unfollow bots, engagement pods or scraping. Use only official Meta APIs. One banned customer account hurts us more than any missing feature.
P7 · Language-first	Telugu, Hindi and English (and code-mixed speech) are first-class from day one — captions, voiceovers, replies, UI copy.
P8 · Transparent and compliant	Customers own their content. Model-training use is opt-in. Pricing, renewal and credit rules are plain. Data handling follows India's DPDP Act 2023 and Meta platform policy.
P9 · Our brand only in the UI	Underlying provider names (voice platform, AI model vendors, etc.) MUST NOT appear anywhere user-facing — UI strings, toasts, emails, widget labels. Backend code may reference them.

SECTION 04

4. Product structure

Creator needs are a subset of business needs. We build one shared core that both audiences use, and a business layer on top. One codebase, one engine — the audience chosen at sign-up only changes defaults, templates and metrics.

4.1 Module map

ID	Module	Creator	Business	Wave
SHARED CORE				
M1	Onboarding and workspaces (creator / business fork)	•	•	1
M2	Brand kit from a website or Instagram link	•	•	1
M3	Content studio: AI posts, stories, captions, calendar	•	•	1
M4	Clip engine: long-form → short-form clips	•	•	1

ID	Module	Creator	Business	Wave
M5	Reels generator and AI video editor	•	•	1
M6	Publisher and scheduler (auto-post)	•	•	1
M7	Instagram DM automation (comment→DM, keyword and AI replies)	•	•	1
M8	Growth analytics	•	•	1
M9	Credits, plans and billing	•	•	1
M10	Approval queue and autopilot	•	•	1
BUSINESS LAYER				
B1	Product photoshoots and business photos/videos	–	•	2
B2	WhatsApp automation and unified inbox	–	•	2
B3	Leads and contacts	–	•	2
B4	Ads engine: launch, campaign analysis, learning loop	–	•	3
B5	AI voice agent (add-on)	–	•	3
B6	Website builder, Google Business Profile, review replies	–	•	4

4.2 Plans and entitlements

Prices are indicative and will be validated with customers; engineering MUST implement plans as configuration (an entitlements table), never as hard-coded checks.

	Creator	Business	Business Pro
Indicative price (₹ / month)	499 – 999	2,999 – 4,999	9,999+
Shared core (M1–M10)	•	•	•
Connected channels	Instagram	Instagram, Facebook, WhatsApp	All + ad accounts
Product photoshoots (B1)	–	•	•
WhatsApp + unified inbox (B2), leads (B3)	–	•	•
Ads engine (B4)	–	–	•
Voice agent (B5)	–	–	Add-on
Team seats	1	3	10
Monthly credits	Config	Config	Config

4.3 How the creator / business choice changes the app

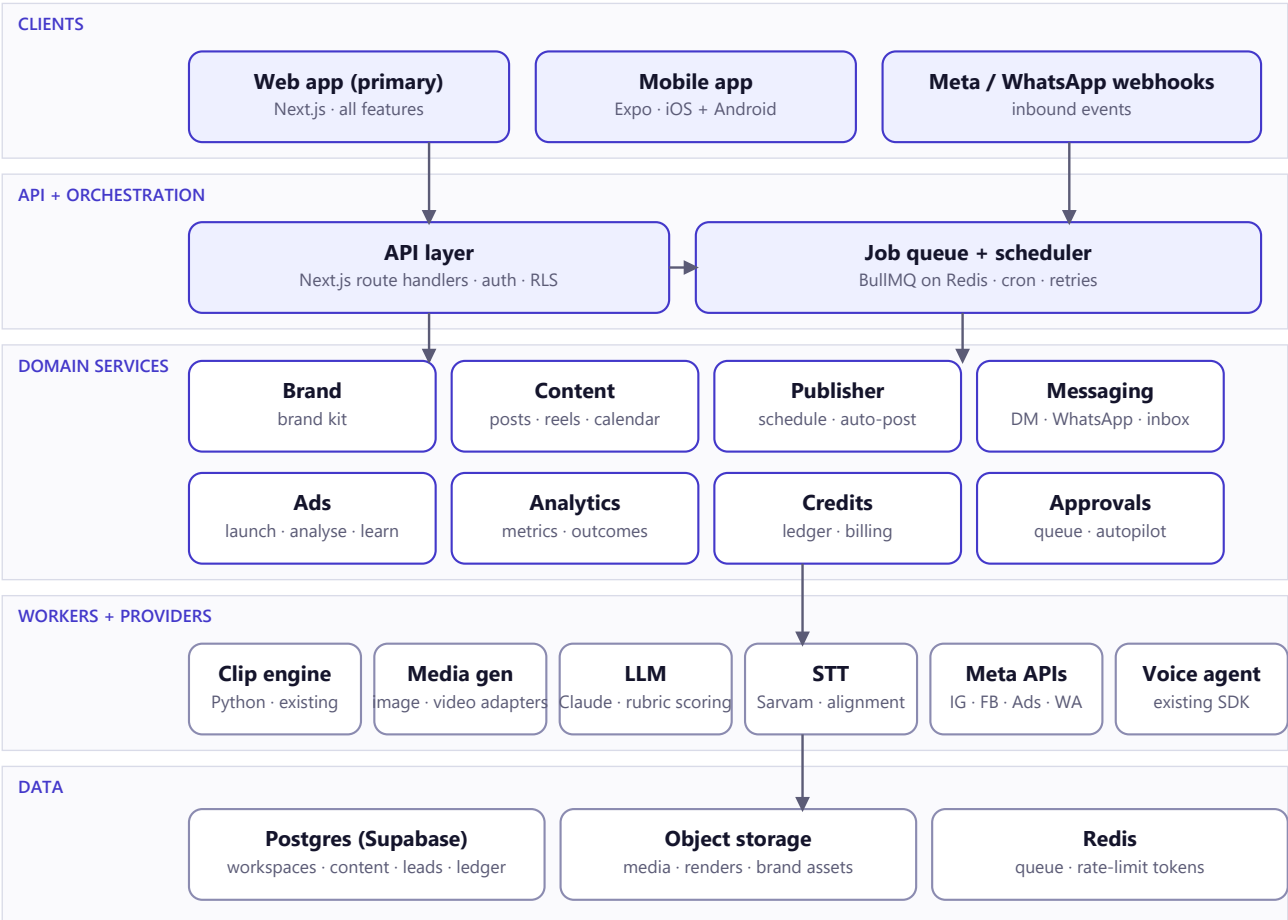
Area	Creator workspace	Business workspace
Home dashboard	Followers, reach, top hooks, posting streak	Leads, cost per lead, ROAS, response time
Content templates	Talking-head clips, trends, educational carousels	Offers, product showcases, testimonials, festival campaigns
Navigation	Studio · Calendar · Clips · Inbox (Instagram) · Growth	Adds Products · Leads · Ads · WhatsApp

Area	Creator workspace	Business workspace
Upgrade path	"Start selling" prompt converts the workspace to Business without data loss	—

SECTION 05

5. System architecture

5.1 Overview



5.2 Recommended stack

The stack extends what already exists in the current Zepply app (Next.js 14, TypeScript, Supabase, Razorpay) and the Python clip engine. Items marked *recommended* are open to change before Wave 1 starts (Section 12).

Layer	Choice	Notes
Web app + API	Next.js 14 (App Router), TypeScript, Tailwind	Existing. Route handlers for the API; server components for dashboards.
Auth + database	Supabase Postgres + Supabase Auth	Existing. Row-level security keyed on <i>workspace_id</i> for every tenant table.
Object storage	Supabase Storage (or S3 + CDN)	Media, renders, brand assets. Signed URLs only — no public buckets with user uploads.
Job queue	BullMQ on Redis (<i>recommended</i>)	All generation, rendering, publishing and syncing runs as jobs with retries and progress.

Layer	Choice	Notes
Media workers	Python 3.12 (existing clip engine) + ffmpeg with libass/HarfBuzz	Packaged as a worker service; GPU optional (alignment is 18–28× faster on GPU).
LLM	Claude via Anthropic API	Scripts, captions, replies, hook scoring. Model per task is configurable.
Speech-to-text	Sarvam + local forced alignment	Existing in the clip engine; handles Telugu-English code-mixing.
Image / video generation	Provider adapters (<i>vendor TBD</i>)	Behind one internal interface so vendors can be swapped per quality/cost.
Payments	Razorpay (India); Stripe later for global	Existing Razorpay subscription + webhook routes.
Mobile	Expo / React Native, iOS + Android (<i>from Wave 1</i>)	Companion to the web app; team already uses Expo (Zepply Swipe). See 5.4–5.7.
Observability	Sentry + structured logs + job dashboard	Every job logs provider, duration and cost.

5.3 Architecture rules

- **Everything slow is a job.** API requests never wait on AI generation, rendering or third-party publishing. They enqueue a job and return its id; the client polls or subscribes for progress.
- **Provider adapters.** Every external AI or platform call goes through an internal adapter interface (*generateImage, generateVideo, transcribe, publishPost, sendMessage...*). No vendor SDK is imported outside its adapter.
- **Idempotency.** Publishing, messaging and credit operations take an idempotency key so retries never double-post, double-send or double-charge.
- **Tenant isolation.** Every row that belongs to a customer carries *workspace_id*; RLS policies enforce it. Workers use a service role but MUST filter by workspace explicitly.
- **Webhooks are verified and queued.** Inbound webhooks (Meta, WhatsApp, Razorpay) verify the signature, store the raw event, return 200 fast, and process asynchronously.
- **Cost cascade.** Use cheap signals to filter before calling expensive models (the clip engine already does this: a cheap audio prefilter keeps ~15 regions, then ~2 LLM calls per video regardless of length). Apply the same pattern everywhere.

5.4 Web and mobile apps

Zepply ships as a **web app (primary)** and a **mobile app for iOS and Android (companion)**, both from Wave 1. The **web app is the main product**: every feature, every editor, every setting lives there first and in full. The mobile app covers the daily tasks users do on the go: approving content, replying to messages, getting notified, capturing photos and uploading videos.

Part	Technology	Role
apps/web	Next.js 14 (existing app), TypeScript, Tailwind	Primary product. All features, full editors, calendar, analytics, ads, billing, settings. Also the public marketing site.
apps/mobile	Expo (React Native) + Expo Router, TypeScript	Companion for iOS and Android: approvals, inbox, notifications, camera, uploads, quick create, summary analytics.
packages/types	zod schemas → TypeScript types	One definition of every entity and API payload, used by web, mobile and API.

Part	Technology	Role
packages/api-client	Typed client for the Zepply API	Both apps call the same API the same way; no app talks to the database directly for business data.
packages/core	Plain TypeScript	Shared rules: credit prices and display, per-platform content limits, validation, status labels.
packages/i18n	String catalogues (te, hi, en)	Same Telugu / Hindi / English UI copy on web and mobile.
packages/tokens	Design tokens	Colours, type scale, spacing, radii, so both apps look like one product.
workers/clipper	Python (existing clip engine)	Media worker; unchanged by the web/mobile split.

UI components are **not** shared between web and mobile (DOM vs. native); logic, types, text and design tokens are. NativeWind MAY be used on mobile so styles follow the same Tailwind conventions as web.

5.5 What each app does

Feature	Web (primary)	Mobile (companion)
Sign-up, onboarding, connect channels	Full	Full
Brand kit	Full: create and edit every field	View; quick edits
Approval queue (M10)	Full	Full, mobile-first: swipe to approve / reject / flag AI mistake
Inbox: Instagram DMs, WhatsApp (M7, B2)	Full	Full + push notification on new message / lead
Create post / Reel from a prompt (M3, M5)	Full	Full
Product photos (B1)	Upload and batch	Camera-first: guided capture of the product
Long video upload for clipping (M4)	Full	Background, resumable upload from the phone
Clip editor (word-level trim, captions)	Full	Light: trim and fix a caption word
Reel timeline editor	Full	Light: swap a scene, fix text
Content calendar	Full month planning, drag and drop	Week list, reschedule
Analytics (M8)	Full dashboards and drill-down	Summary cards + weekly digest
Ads manager (B4)	Full	Monitor, pause, approve new campaigns
Website builder, Google profile (B6)	Full	–
Team, roles, workspace settings	Full	View
Billing and credits (M9)	Razorpay	In-app purchase via App Store / Play Store (see 5.6)

5.6 Mobile-specific requirements

- **Payments:** app stores generally require digital subscriptions and credits sold inside the app to use their in-app purchase systems. Use one entitlement service (e.g. RevenueCat) so a plan bought on web (Razorpay) or in the app (store purchase) unlocks the same features everywhere. Store fees (15–30%) affect pricing. Verify current App Store and Play rules for India before launch.
- **Push notifications** (Expo Notifications → APNs / FCM): new message or lead, content waiting for approval, post published or failed, credits low, weekly digest. Users control each type.

- **Large uploads:** resumable uploads (tus protocol, supported by Supabase Storage) that continue in the background and survive network drops; show progress; upload over Wi-Fi only if the user chooses.
- **Login:** Supabase Auth on both apps; Meta account connection through the system browser (OAuth). Meta app review must include the mobile flow.
- **Store compliance:** in-app account deletion, privacy labels / Data safety form, a one-tap “report AI content” control on generated media (Google Play policy for AI-generated content), demo account for reviewers.
- **Releases:** EAS Build and Submit for store builds; EAS Update for over-the-air fixes to JavaScript between store releases.
- **Rendering stays on the server.** The phone previews and edits; all generation and final rendering run in workers, so phones of any price work.
- **Offline:** drafts and the approval queue are cached; actions taken offline sync when the connection returns.

5.7 Rules that keep web and mobile in sync

- **One API for every client.** All business data goes through versioned REST endpoints (*/api/v1/...*) with bearer-token auth. Web pages MUST NOT rely on server-only data paths that mobile cannot use for the same feature.
- **Web first, mobile follows.** A feature is designed and shipped on web; the mobile version (full or light, per 5.5) follows in the same or next release. Mobile never gets a feature web does not have.
- **Same words, same states.** Status names, credit costs and error messages come from *packages/core* and *packages/i18n*, never hard-coded in an app.
- **Backwards-compatible API.** Old mobile versions stay in use for weeks; API changes are additive, and a minimum supported app version can be enforced with an in-app update prompt.

SECTION 06

6. Data model

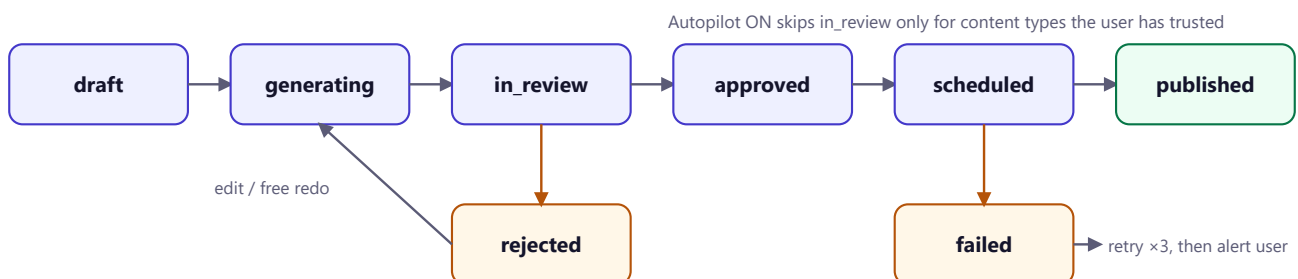
Core entities below. Field lists show the important columns, not every column. All tables include id (uuid), created_at, updated_at; tenant tables include workspace_id.

6.1 Entities

Entity	Key fields	Notes
workspace	type (creator business), name, plan_id, locale, primary_languages[], timezone, autopilot_settings (json)	The tenant. Type drives defaults only.
member	workspace_id, user_id, role (owner admin editor viewer)	Team seats per plan.
channel_account	platform (instagram facebook whatsapp meta_ads youtube), external_id, handle, access_token (encrypted), token_expires_at, scopes[], status	Connected accounts. Tokens encrypted at rest; refresh job runs daily.
brand_kit	logo_asset_id, colors[], fonts[], voice_tone, audience, products (json), dos[], donts[], sample_posts[], source_urls[], version	Versioned; every generation records the brand_kit version it used.
asset	kind (image video audio doc), storage_path, mime, duration, width, height, source (upload generated imported), product_id?	Uploaded and generated media.
product	name, description, price, images[] (asset ids), tags[]	Business catalogue; used by photoshoots and ads.

Entity	Key fields	Notes
content_item	type (post carousel story reel clip), status, caption, hashtags[], language, media[] (asset ids), brand_kit_version, source_asset_id?, scheduled_at, published_at, channel_targets[], external_post_ids (json)	The central object. State machine in 6.2.
generation_job	content_item_id?, kind, provider, input (json), output (json), status, progress, credits_charged, cost_inr, error, is_free_redo	Every AI call. Basis for cost analytics.
calendar_slot	date, time, channel, content_item_id?, theme	The always-on calendar; filler job keeps 14 days ahead.
automation_rule	channel, trigger (comment_keyword dm_keyword new_follower story_reply any_message), match (json), action (send_dm ai_reply tag_contact handoff), template, active, stats (json)	DM / WhatsApp automations.
contact	channel identities (json), name, phone?, language, tags[], stage (new engaged lead customer), consent (json)	People who interact. Becomes a lead in Business.
conversation / message	contact_id, channel, status (open bot human closed) / direction, body, media, sent_by (ai rule human), external_id	Unified inbox.
ad_campaign / ad_creative	platform, objective, budget, status, external_ids / content_item_id, variant, metrics	Business Pro (Wave 3).
metric_snapshot	object_type, object_id, metric, value, captured_at	Time-series of reach, engagement, followers, spend.
outcome_event	contact_id?, campaign_id?, kind (lead qualified booked sale), value_inr, source	What actually happened; drives the learning loop.
credit_ledger	delta, reason, generation_job_id?, balance_after, idempotency_key	Append-only. Balance = sum of deltas.
subscription	plan_id, status, provider (razorpay), current_period_end, provider_ids	Billing state from webhooks.
approval	content_item_id, decided_by, decision (approve reject edit), reason, ai_mistake (bool)	Feeds quality metrics and free redos.
device	user_id, platform (ios android web), push_token, app_version, notification_prefs (json), last_seen_at	Push notifications and minimum-version checks.

6.2 Content item lifecycle



- **draft → generating:** a generation job is enqueued; credits are reserved (not spent) at this point.
- **generating → in_review:** job succeeds; reserved credits are committed. On job failure credits are released and the item returns to draft.
- **in_review → rejected:** user rejects. If flagged "AI mistake", the next redo is free (P3).
- **approved → scheduled → published:** publisher job posts at *scheduled_at*. On platform error it retries 3× with backoff, then marks *failed* and notifies the user. Never retry a post that may have succeeded without checking

the platform first (idempotency).

SECTION 07

7. Wave 1 — shared core module specs

Wave 1 ships the Creator plan and the foundation every later wave builds on. Each module lists user stories, functional requirements, acceptance criteria and failure handling. A module is done when every acceptance criterion passes on production data.

M1 · Onboarding and workspaces

Wave 1 · Creator + Business

Purpose. Get a new user from sign-up to their first approved piece of content in under 10 minutes.

User stories

- As a new user, I pick “I’m a creator” or “I’m a business” so the app is set up for my goals.
- As a user, I connect Instagram (and for business, Facebook and WhatsApp) in one guided step.
- As an owner, I invite teammates and set their role.

Functional requirements

- Sign-up with email OTP and Google; phone OTP SHOULD be supported for India.
- Audience fork at step 1; stores *workspace.type* and applies defaults (dashboard, templates, nav — Section 4.3).
- Language step: pick primary content languages (multi-select: Telugu, Hindi, English, others) and UI language.
- Channel connection via official Meta login; show exactly which permissions are requested and why.
- Onboarding ends by generating 3 sample posts from the brand kit (M2) — free, no credits.
- Creator → Business upgrade converts the workspace in place with no data loss.

Acceptance criteria

- A new user reaches a reviewed sample post in ≤ 10 minutes (median, measured).
- Skipping channel connection is allowed; the user can still generate and download content.
- Role permissions enforced server-side (viewer cannot approve or spend credits).

Edge cases and failure handling

- Meta login cancelled or permissions partly granted → show which features are unavailable and a reconnect button.
- Instagram personal (non-professional) account → explain how to switch to a Professional account; block auto-posting until done.

M2 · Brand kit from a link

Wave 1 · Creator + Business

Purpose. Learn the brand once from a website or Instagram link so every output is on-brand without re-explaining.

User stories

- As a user, I paste my website or Instagram handle and get a brand kit in about a minute.
- As a user, I edit anything the AI got wrong (logo, colours, tone, audience, products).

Functional requirements

- Inputs: website URL, Instagram handle (via connected account), or manual upload of logo and sample posts.
- Extract: logo, colour palette, fonts (best effort), tone of voice, audience description, products/services, content themes, languages used.
- Website: fetch and parse public pages (home, about, products) server-side; respect robots.txt; no login-walled scraping.
- Instagram: read the connected account's own recent media and captions via the official API only.
- Show results as an editable card with a confidence hint per field; every field user-editable.
- Brand kit is versioned; edits create a new version; generations record the version used.

Acceptance criteria

- Brand kit generated in ≤ 90 s for a typical site.
- User edits take effect on the very next generation.
- Logo is never altered by AI — stored and composited as uploaded/extracted.

Edge cases and failure handling

- Site unreachable / heavy JS → fall back to Instagram or manual input with a clear message.
- Multiple logos found → let user choose.

Implementation notes

- Store the raw extraction separately from the user-edited kit so re-extraction never overwrites user edits.

M3 · Content studio — AI posts, stories and calendar

Wave 1 · Creator + Business

Purpose. Keep a 14-day content calendar filled with on-brand posts, carousels and stories, ready for approval.

User stories

- As a user, I see a monthly calendar already planned with themes and posts.
- As a user, I create a post from a prompt, a photo, or a template, in my language.
- As a user, I edit one line of a caption or swap one image without regenerating the whole post.

Functional requirements

- Calendar planner job: generates a monthly plan (themes, formats, best times) from brand kit, audience type, festivals/events calendar for India, and past performance (M8).
- Filler job keeps calendar slots filled ≥ 14 days ahead (respecting plan limits).
- Post types: single image, carousel (up to 10), story, text-on-image. Output sizes per platform (1:1, 4:5, 9:16).
- Caption writer: language and code-mixing per workspace setting; hashtags; CTA; character limits per platform.
- Part-level editing: regenerate caption only, image only, a single carousel slide, or one text element (P2).
- Festival and event packs (Diwali, Sankranti, Ugadi, Ramzan, etc.) as templates.
- Every generate/regenerate shows credit cost and asks for confirmation (P4).

Acceptance criteria

- Single post generated in ≤ 30 s; carousel in ≤ 60 s.
- Telugu and Hindi text renders correctly (conjuncts shaped, no broken glyphs) in all image outputs.
- Calendar never shows an empty slot within the next 7 days while credits remain.

Edge cases and failure handling

- Generation fails → credits released, item returns to draft with a readable error.
- Out of credits → filler pauses, user notified; nothing is generated silently on overage.

M4 · Clip engine — long-form to short-form

Wave 1 · Creator + Business

Purpose. Turn podcasts, interviews, lives and walkthrough videos into ranked, captioned, vertical short clips.

User stories

- As a podcaster, I upload a 1-hour episode (or paste a YouTube link I own) and get the best 10–15 clips ranked.
- As a user, I adjust a clip's start/end, fix a caption word and pick a caption style, then publish.

Functional requirements

- Ingest: file upload (resumable, up to 4 h / 10 GB) and URL import for content the user owns.
- Pipeline (exists in the Python clip engine): transcribe (Sarvam) → forced word alignment → cheap audio/timing prefilter → LLM hook scoring against the rubric → boundary refinement → reframe to 9:16 with speaker tracking → burned captions → render.
- Return a ranked list with score, hook reason, duration and preview; user selects which to render in full quality.
- Editor: trim boundaries on a word-level transcript, edit caption text, choose style, choose headline overlay.
- Captions: Telugu script, romanized, or English — per user choice; code-mixed speech supported.
- Rejected clips and their reasons are logged as training data for rubric/prefilter tuning.

Acceptance criteria

- 1-hour source returns ranked candidates in ≤ 15 min (GPU) / ≤ 40 min (CPU) with live progress.
- LLM cost per source stays roughly flat regardless of source length (cascade preserved).
- Caption shaping test (existing *test_telugu_shaping.py*) passes on the worker image before deploy.

Edge cases and failure handling

- Multi-camera podcasts: speaker-to-face matching is position-based today and can mis-assign when both speakers are centre-framed — surface a “switch speaker” control; appearance-based matching is a planned improvement.
- Source without speech → explain that clipping needs speech; offer Reels generator instead.

Implementation notes

- Wrap the existing CLI as a worker that consumes jobs; do not rewrite it. Keep the rubric file owned by the product team and hashed so edits invalidate cached scores (existing behaviour).

M5 · Reels generator and AI video editor

Wave 1 · Creator + Business

Purpose. Create Reels from a description, photos or clips — script, scenes, voiceover, captions and edit — in the user's language.

User stories

- As a user, I describe what I want ("3 reasons to visit my salon this Diwali") and get a ready Reel.
- As a user, I change one scene, re-record the voiceover line, or swap music without starting over.

Functional requirements

- Inputs: prompt, user photos/videos, products, or a clip from M4.
- Script writer produces hook, scenes and CTA; user can edit script before rendering (cheap step before expensive step).
- Scene builder uses user media first, generated media second; product shots obey P1.
- Voiceover in Telugu, Hindi, English (and code-mixed) with selectable voices; captions auto-timed.
- Timeline editor: per-scene replace/trim/reorder, text overlays, music (licensed library only), transitions.
- Multiple hook variants (A/B) for the same Reel at reduced credit cost.

Acceptance criteria

- Script ready in ≤ 20 s; full Reel render ≤ 5 min with progress.
- Editing one scene re-renders only what is needed and costs only that scene's credits.
- No unlicensed music is ever available in the library.

Edge cases and failure handling

- Generated scene contradicts the product/prompt → user flags "AI mistake" → free redo of that scene (P3).

M6 · Publisher and scheduler

Wave 1 · Creator + Business

Purpose. Publish approved content to connected channels at the right time, reliably, without the user opening the app.

User stories

- As a user, I approve a week of content once and it goes out on its own.
- As a user, I see what was posted, what failed and why.

Functional requirements

- Channels in Wave 1: Instagram (feed, carousel, Reels, stories where API allows) and Facebook Page. YouTube Shorts SHOULD follow.
- Best-time scheduling from the account's own audience activity (M8), with manual override.
- Per-channel validation before scheduling (aspect ratio, duration, caption length, media count).
- Respect platform rate limits (e.g. Instagram API publishing caps per 24 h) with a visible quota.
- Status tracking with external post ids; deep link to the live post.
- Pause-all switch per workspace (for emergencies or brand events).

Acceptance criteria

- $\geq 99\%$ of scheduled posts publish within 5 minutes of their time (excluding platform outages).
- No duplicate posts under retries (idempotency verified by test).
- Token expiry is detected ≥ 3 days ahead and the user is prompted to reconnect.

Edge cases and failure handling

- Platform rejects media → mark failed with the platform's reason in plain language and a fix suggestion.

- Channel disconnected → hold queue, notify, resume automatically after reconnect.

M7 · Instagram DM automation

Wave 1 · Creator + Business

Purpose. Reply to comments and DMs at any hour — keyword automations (comment→DM) and AI replies that sound like the brand.

User stories

- As a creator, when someone comments "LINK" on my Reel, they automatically get a DM with the link.
- As a user, common DM questions get answered instantly in the sender's language, and I take over when needed.

Functional requirements

- Rule builder: trigger (comment keyword on specific/all posts, DM keyword, story reply, new message) → action (private reply DM, public comment reply, AI reply, tag contact, hand off to human).
- AI reply brain uses brand kit + a workspace FAQ/knowledge base; detects language and replies in it; never invents prices, offers or facts not in the knowledge base — hands off instead.
- Inbox (Instagram channel in Wave 1): conversation list, bot/human status, take-over and give-back controls.
- Respect Meta messaging rules: replies only within the allowed messaging window; one private reply per comment; no unsolicited DMs.
- Per-rule stats: triggers, messages sent, link clicks.

Acceptance criteria

- Keyword comment → DM delivered in ≤ 60 s (p95) when within platform rules.
- AI reply is suppressed and handed off when confidence is low or the topic is outside the knowledge base.
- A human reply always pauses the bot for that conversation until given back.

Edge cases and failure handling

- Message outside the allowed window → no send; conversation flagged for human.
- Webhook burst (viral post) → queue absorbs; replies keep order per conversation; rate limits respected.

Implementation notes

- Build as the shared messaging engine: the same rules/AI/inbox code serves WhatsApp in Wave 2 with a different channel adapter.

M8 · Growth analytics

Wave 1 · Creator + Business

Purpose. Show what is working and feed results back into planning.

User stories

- As a creator, I see follower growth, reach and which hooks/topics perform best.
- As a user, I get a weekly summary with 3 concrete suggestions.

Functional requirements

- Daily sync of account and media insights from connected channels into *metric_snapshot*.
- Dashboards per Section 4.3; post-level drill-down; compare formats, topics, languages, posting times.
- Hook/topic performance model feeds M3 planner and M5 hook variants.
- Weekly digest (in-app + email/WhatsApp) with insights and next-week plan changes.

Acceptance criteria

- Dashboards load in ≤ 2 s for 12 months of data.
- Planner demonstrably uses performance data (logged reason on each planned item).

Edge cases and failure handling

- Insight API unavailable for an account type → show what is available, explain the gap.

M9 · Credits, plans and billing

Wave 1 · Creator + Business

Purpose. Fair, transparent usage-based billing that never surprises the user.

User stories

- As a user, I always know my balance and what an action will cost before I do it.
- As a user, I get a free redo when the AI made a mistake.

Functional requirements

- Plans and entitlements stored as configuration (Section 4.2).
- Append-only credit ledger with reserve → commit → release flow tied to generation jobs.
- Price list per action (post, carousel, Reel per scene, clip render, photoshoot image) stored as configuration.
- Free redo: "AI mistake" flag on rejection grants one free regeneration of that item/part; limit N per month per plan to prevent abuse.
- Razorpay subscriptions (existing routes) + one-time credit packs; invoices with GST.
- Clear renewal date, cancel anytime from settings, no forced subscription to buy a pack.

Acceptance criteria

- Ledger balance always equals sum of deltas (nightly reconciliation job, alert on mismatch).
- No code path spends credits without a confirmation token from the client (except autopilot within a user-set monthly budget).
- Failed jobs never leave credits deducted.

Edge cases and failure handling

- Payment webhook delayed → keep previous entitlements until confirmed; never lock content the user already created.

M10 · Approval queue and autopilot

Wave 1 · Creator + Business

Purpose. Let users stay in control at first and hand over to Zepply as trust grows — the switch that makes Zepply always-on.

User stories

- As a user, I review everything in one queue: approve, edit, or reject with a reason.
- As a user, once I trust Zepply for a content type, I switch on autopilot for it with a monthly credit budget.

Functional requirements

- Single queue across posts, Reels, clips and (later) ad creatives; swipe-style mobile-friendly review.
- Autopilot per content type and per channel, with a monthly credit cap and quiet hours.
- Suggest enabling autopilot when the approval rate for a type is $\geq 80\%$ over the last 20 items.
- Autopilot actions are logged and reversible (unpublish/delete where the platform allows).

Acceptance criteria

- With autopilot off, nothing is published or sent without approval.
- With autopilot on, spend never exceeds the user-set cap.

Edge cases and failure handling

- Brand-safety check fails on an autopilot item (sensitive topic, competitor mention, low confidence) → route to queue instead of publishing.

SECTION 08

8. Business layer specs (Waves 2–4)

Specified at feature level. Each will get a full spec like Section 7 before its wave starts.

B1 · Product photoshoots and business photos/videos (Wave 2)

- Upload phone photos of a product → background removal → composite onto generated scenes (studio, lifestyle, festive, model). **The product itself is never regenerated (P1).**
- Fashion: on-model and virtual try-on MUST preserve garment design, colour, sleeves and pattern; a fidelity check compares product region against the source and blocks outputs that drift.
- Batch mode for catalogues (Amazon, Flipkart, Shopify, Meesho size presets); upscaling; background change.
- Business videos: product showcase, testimonial, offer and walkthrough templates using M5.

B2 · WhatsApp automation and unified inbox (Wave 2)

- WhatsApp Business Platform (Cloud API) channel adapter on the shared messaging engine from M7.
- Unified inbox: Instagram + Facebook + WhatsApp (+ voice call summaries in Wave 3) per contact.
- Template messages for outside the 24-hour customer-service window (Meta-approved templates only); broadcast to opted-in contacts with consent records.
- Click-to-WhatsApp ads land in the same inbox (connects to B4).

B3 · Leads and contacts (Wave 2)

- Every engaged person becomes a *contact*; stages new → engaged → lead → customer; AI qualification questions per business type.
- Booking: appointment/visit scheduling with reminders on WhatsApp.
- Outcome events (qualified, booked, sale) recorded — the fuel for the ads learning loop.

B4 · Ads engine: launch, campaign analysis, learning loop (Wave 3)

- Launch Meta campaigns (Instagram/Facebook) from approved creatives: objectives traffic, leads, messages (Click-to-WhatsApp), sales.
- Guard-rails: daily budget caps, spend alerts, auto-pause rules; user approves every new campaign.
- Campaign analysis: CPL, CTR, ROAS, frequency, creative fatigue; plain-language weekly report.
- **Learning loop:** send qualified/booked/sale outcomes back to Meta via the Conversions API so delivery optimises for real customers, not form-fills; use winning hooks/creatives to generate the next variants automatically.
- Google Ads SHOULD follow after Meta is stable.

B5 · AI voice agent add-on (Wave 3)

- Reuse the existing Zepply voice agent: instant call-back to new leads, inbound call answering, qualification and booking, in Telugu, Hindi or English.
- Call summaries and outcomes flow into the unified inbox and outcome events.
- Provider branding never visible to users (P9).

B6 · Website, Google Business Profile, reviews (Wave 4)

- Website builder from the brand kit (landing pages first); lead forms connect to B3.
- Google Business Profile: posts, photos, Q&A; AI review replies with approval.
- These are commodity features — build lightly or integrate rather than invest heavily.

SECTION 09

9. Integrations and external dependencies

9.1 Platform permissions

Exact permission names and limits change; verify against current Meta documentation at implementation time.

Integration	Used by	Typical permissions / access
Instagram API (Instagram Login)	M2, M6, M7, M8	instagram_business_basic, instagram_business_content_publish, instagram_business_manage_messages, instagram_business_manage_comments
Facebook Pages / Login for Business	M6, M8	pages_show_list, pages_read_engagement, pages_manage_posts, business_management
WhatsApp Business Platform (Cloud API)	B2, B3	whatsapp_business_messaging, whatsapp_business_management; verified business; approved templates
Meta Marketing API + Conversions API	B4	ads_management, ads_read (Advanced Access); dataset/pixel for outcome events
YouTube Data API	M4 import, M6 (later)	OAuth upload scope; quota application for volume
Google Business Profile API	B6	Access application required

Start these approvals in week 1 — they take weeks and block Waves 1–3

1. Meta Business Verification for the Zepply business.
2. Meta App Review for Instagram messaging, comments and content publishing (needs screencasts of working features — build a review build early).
3. WhatsApp Business Platform onboarding (direct Cloud API or via a BSP) and display-name approval.
4. Marketing API Advanced Access.
5. Google Business Profile API access (only needed for Wave 4, but the queue is long).

9.2 AI and media providers

Capability	Current / candidate	Adapter rule
LLM (scripts, captions, replies, scoring)	Claude (Anthropic API) — already used by the clip engine	Model per task configurable; log tokens and cost per job
Speech-to-text	Sarvam + local forced alignment (existing)	Language/code-mix aware; word timestamps required
Text-to-speech / voiceover	TBD — must support Telugu, Hindi, English	Voice catalogue stored in our DB, not vendor names in UI
Image generation / editing	TBD — evaluate on product fidelity + Indic text	Product region always composited, never generated (P1)
Video generation	TBD — evaluate per quality / cost per second	Used for scenes only; user media preferred

Capability	Current / candidate	Adapter rule
Background removal / upscaling	TBD (self-host or API)	Deterministic; no credit charge for failures
Voice agent	Existing voice SDK in current app	Already wrapped server-side; extend, don't replace

SECTION 10

10. Non-functional requirements

10.1 Performance targets

Operation	Target
Sign-up to first reviewed sample post	≤ 10 min (median)
Brand kit from link	≤ 90 s
Single post / carousel	≤ 30 s / ≤ 60 s
Reel script / full render	≤ 20 s / ≤ 5 min
Clip candidates from 1-hour source	≤ 15 min GPU, ≤ 40 min CPU
Comment→DM / AI reply latency	≤ 60 s p95
Scheduled post accuracy	≥ 99% within 5 min
Dashboard load (12 months)	≤ 2 s

10.2 Security

- OAuth tokens and API keys encrypted at rest (pgcrypto or KMS); never logged; never sent to the client.
- Row-level security on every tenant table; automated tests that one workspace cannot read another's rows.
- Webhook signature verification for Meta, WhatsApp and Razorpay; replay protection.
- TLS verification always on. If a development machine's antivirus intercepts HTTPS, fix the trust store (e.g. truststore) — **never** disable verification (verify=False).
- Signed, expiring URLs for all media; user uploads never in a public bucket.
- Least-privilege service roles for workers; secrets in the deployment secret store, not in the repo.

10.3 Privacy and compliance

- India DPDP Act 2023: consent records, purpose limitation, data export and deletion on request, breach process.
- Meta Platform Terms and WhatsApp Business policy: consent for marketing messages, messaging windows, data deletion callback.
- Terms of service MUST state: customer owns inputs and outputs; training on customer data is opt-in; auto-renewal and cancellation terms; refund rules including free redos for AI mistakes; governing law India.
- Advertising rules: no misleading claims; real-estate content respects RERA advertising requirements.

10.4 Cost control

- Every generation job records provider, units and cost in INR; dashboards show gross margin per workspace and per feature.
- Cheap-before-expensive cascades (script before render, preview before full quality, prefilter before LLM).
- Per-workspace and global spend caps with alerts; kill-switch per provider.

- Cache deterministic results (transcripts, alignments, embeddings) keyed by input hash.

10.5 Languages and text rendering

- Telugu, Hindi and English supported in captions, voiceovers, replies and UI from Wave 1; architecture allows adding Tamil, Kannada, Marathi, Bengali later.
- Rendering pipelines MUST use HarfBuzz shaping (ffmpeg with libass + harfbuzz + fribidi). Without it, Telugu conjuncts break silently.
- Fonts validated for script coverage before use (existing style check in the clip engine).
- Language detection on inbound messages; reply language follows the sender.

10.6 Reliability and observability

- Jobs: retries with exponential backoff, dead-letter queue, manual re-run from an admin screen.
- Structured logs with workspace_id and job_id; error tracking; uptime monitoring for webhooks.
- Daily backups with tested restore; staging environment with its own Meta test app and channels.

SECTION 11

11. Roadmap and success metrics

11.1 Waves and exit criteria

Wave	Scope	Ships	Exit criteria (before next wave)
0 · Setup (2 weeks)	Monorepo (web, mobile, shared packages), environments, auth, workspaces, RLS, job queue, adapters skeleton, Expo app shell; start Meta approvals and store developer accounts	Internal	Staging live; approvals submitted
1 · Shared core	M1–M10 on web; mobile companion (approvals, inbox, notifications, create, uploads)	Creator plan (web + iOS/Android)	20+ active creators; draft approval rate ≥ 60%; publish success ≥ 99%; Meta app review approved
2 · Business	B1, B2, B3	Business plan	10+ paying businesses; WhatsApp live; product-fidelity rejections < 5%
3 · Growth engine	B4, B5	Business Pro	Outcome events flowing to Conversions API; measurable CPL improvement on pilot accounts
4 · Presence	B6 + remaining mobile “light” features	All plans	—

Release rule

A wave does not ship until its quality bar is met. Breadth before quality is the main failure mode in this market.

11.2 Metrics to instrument from day one

Metric	Definition	Why
North star	Weekly active workspaces with ≥ 3 published items and ≥ 1 automated engagement	Captures the Create → Reply → Multiply loop actually running
Time to first value	Sign-up → first approved item	Onboarding quality
Draft approval rate	Approved ÷ reviewed AI drafts	Content quality
AI-mistake rate	Items flagged “AI mistake” ÷ generated	Fidelity and trust (P1, P3)

Metric	Definition	Why
Autopilot adoption	% workspaces with ≥ 1 autopilot type on	Trust earned
Response time	Median seconds to first reply on DMs / WhatsApp	Engagement promise
Gross margin	Revenue – provider cost, per workspace and per feature	Unit economics
Retention	Monthly logo churn; creator → business upgrades	Business health

Instrument these as product events (e.g. *workspace_created*, *brand_kit_ready*, *item_generated*, *item_approved*, *item_rejected*, *ai_mistake_flagged*, *item_published*, *publish_failed*, *dm_auto_replied*, *handoff_to_human*, *autopilot_enabled*, *credits_purchased*) with *workspace_id*, *plan* and *audience type* on every event.

SECTION 12

12. Existing assets and open decisions

12.1 What already exists and should be reused

Asset	Where	Reuse in
Zepply web app — Next.js 14, Supabase auth, dashboard, leads, campaigns, reports, settings, Razorpay billing routes	Desktop/Zepply - Copy (app/, lib/)	M1, M9, B3 — base for the platform shell
Voice agent integration — agents, calls, recordings, phone numbers, dispatch, campaigns	app/api/*, lib/, voice-engine/	B5
Clip engine — ingest, Sarvam transcription, forced alignment, prefilter cascade, rubric scoring with Claude, boundary refinement, 9:16 reframe, Telugu captions and styles, render	Desktop/CLI/clipper (Python)	M4, parts of M5 (captions, render)
Hook rubric tuned on real Telugu examples + rejects log for tuning	Desktop/CLI/prompts, rejects.jsonl	M4, M5 hook variants
Expo / React Native experience	Zepply Swipe app	Mobile app (from Wave 1)

Housekeeping before Wave 1

- The clip engine currently lives inside a git repo rooted at the user home folder — give it its own repository.
- Demo-only code (seed routes, client demo flags) should be isolated from production paths.
- Decide monorepo vs. separate repos (web app, workers) — see below.

12.2 Open decisions

#	Decision	Options	Recommendation
D1	Repository layout	Monorepo vs. separate repos	Monorepo (Turborepo + pnpm): apps/web, apps/mobile, shared packages; Python worker as its own package
D2	Job queue	BullMQ/Redis vs. Postgres-based queue	BullMQ/Redis for throughput and scheduling
D3	Image/video generation vendors	Evaluate 2–3 per capability	Bake-off on product fidelity + Telugu text before choosing
D4	WhatsApp access	Direct Cloud API vs. a BSP	Direct Cloud API unless onboarding speed requires a BSP
D5	Final pricing and credit prices	Indicative in 4.2	Validate with 10–20 pilot users in Wave 1

#	Decision	Options	Recommendation
D6	Mobile payments	Store in-app purchase vs. web-only purchase	Entitlement service (e.g. RevenueCat): store purchase in app, Razorpay on web, one set of entitlements

SECTION 13

13. Glossary

Term	Meaning
Autopilot	Mode where approved content types publish/send without per-item approval, within a credit budget
Brand kit	Learned and user-edited brand profile: logo, colours, fonts, tone, audience, products
Cascade	Running cheap filters first so expensive AI calls only see the best candidates
Clip engine	Existing Python pipeline that turns long-form video into ranked short clips
Code-mixed	Speech or text mixing languages, e.g. Telugu-English (Telugish) or Hindi-English (Hinglish)
Comment→DM	Automation that sends a private DM when someone comments a keyword
Conversions API	Meta API for sending real outcomes (leads, sales) back to optimise ad delivery
Credit	Unit of AI usage; reserved when a job starts, committed on success, released on failure
Free redo	Regeneration at no credit cost when the user flags an AI mistake
Outcome event	A real-world result (qualified lead, booking, sale) linked to content or campaigns
Workspace	One creator or business tenant, with its members, channels and data
IAP	In-app purchase through the App Store / Play Store billing systems
OTA update	Over-the-air update of the mobile app's JavaScript without a new store release (EAS Update)

End of document. Changes to principles (Section 3) or plan entitlements (Section 4.2) require product-owner sign-off.